

## 5 — Proxy: traps that fire on every read and write

### OBJECTS TRAPS INTERCEPTION

**Q. `claps` was never defined on the object, so why does `article.claps = 5` get logged?**

**ANSWER.** Because `article` is not the real object. It is a proxy wrapping a target, with a handler that intercepts every operation. Writing `article.claps = 5` fires the `set` trap before anything reaches the target, and the trap logs the access, even for a property that did not exist a moment ago.

**WHY IT WORKS.** A getter guards one property you named in advance. A proxy sits in front of the *entire* object. Every read runs the `get` trap and every write runs the `set` trap, no matter which key is touched, no matter whether it existed before. Your code runs in the gap between the access and the target, every single time.

examples/proxy.js

```
const target = { headline: "Markets rally" };
const article = new Proxy(target, { // wraps
  target in a proxy
  get(t, k) { // get trap -
    runs on every read
    console.log("read:", k);
    return t[k];
  },
  set(t, k, v) { // set trap -
    runs on every write
    console.log("write:", k);
    t[k] = v;
    return true;
  }
});
article.headline; // fires get
→ "read: headline"
article.claps = 5; // fires set
→ "write: claps"
```

## THE CHECKPOINT

1 · WRITE

```
article.claps = 5
```



2 · TRAP

```
set(t, "claps", 5)
```

LOGS "WRITE: CLAPS"



3 · TARGET

```
target.claps = 5
```

A BRAND-NEW PROPERTY

- ① **Summary.** A proxy intercepts every read and write through traps, even on properties that did not exist a moment ago. We can now run code on every access — the open question is what the most useful thing to run there would be.